

CCDSALG MCO2 - Graph Implementation

(Linked List, Stack, Queue, Heap, and Graph Application)

Term 3 AY 2025 - 2026

Tristan Catera - tristan_catera@dlsu.edu.ph
Ken Latido - ken_latido@dlsu.edu.ph
Alek Medran - alek_medran@dlsu.edu.ph
S03 - Group 18

2401 Taft Avenue
Manila, Philippines

ABSTRACT

The project creates an undirected weighted graph through the use of adjacency lists, thus showing an application of important data structures, such as stacks, queues, or linked lists. The software's functionality is represented by using a graph, allowing for the process of adding edges and vertices. Moreover, the users could check edge existence or vertex degree, as well as run simple graph traversal methods – BFS or DFS. In addition, an implementation of algorithms allows for checking if two points are connected on the graph as well as the generation of the Minimum Spanning Tree (MST) using Prim's Algorithm.

INTRODUCTION

Graphs are common terms used in computer science representing a relationship between various interconnected items like cyberspace systems, transport networks, social networks and many more. In the project, undirected weighted graphs using adjacency list representation are implemented and other auxiliary data structures like stack and queue are applied for graph traversals and checking paths. Moreover, the program works with basic

graph operations, such as inserting vertices and edges, calculating degrees, checking edges, traversing graphs, checking path existence, visualizing graphs and generating Minimum Spanning Tree of the graph.

STACK, QUEUE, AND HEAP IMPLEMENTATION

In its execution of different graph-related tasks, the project relies on stack and queue methods to represent graphs through adjacency lists. In relation to the stack, being a Last-In-First-Out (LIFO) structure, it comes into play when performing Depth First Search (DFS) since it allows storing nodes in the course of the DFS as it explores graph paths. The queue is a First-In-First-Out (FIFO) structure and is utilized in the implementation of Breadth-First Search (BFS) and verifying if a certain path exists in the graph since the queue allows storage of nodes in the order of their discovery. Traditionally, heaps are known to improve efficiency of certain graph algorithms, including that of Prim, but this scheme does not use heaps. Instead, the

Minimum Spanning Tree (MST) of the graph is built based on edges one at a time.

Stack Implementation

A stack was created with an array and works according to the Last-In, First-Out (LIFO) fundamentals. The stack has basic operations push, pop, peek, and checking whether the stack is empty. All of them have constant execution time. In general, stack is used while implementing Depth-First Search (DFS), which guarantees that the vertices will be stored and processed in temporary storage locations while some given path is finished before going back to the last visited vertices.

Queue Implementation

The queue uses an array and follows the First-In, First-Out (FIFO) principle. It has enqueue, dequeue, and empty-check operations that work in constant time. Queues are used in Breadth-First Search (BFS) and path existence checking in which vertices are examined level by level and all reachable vertices are visited in the correct order.

Heap Implementation

A heap module was prepared as part of the project structure. However, the final submitted version does not use the heap in its active program flow. The Minimum Spanning Tree is generated without a heap-based priority queue.

GRAPH IMPLEMENTATION

The graph is built as an undirected weighted graph with the use of an adjacency list. Each vertex holds a name that is unique and a linked list of neighboring vertices, while each edge contains the information about the destination and its weight in terms of cost. Adjacency lists enable effective insertion, traversal, and searching of neighboring vertices but require less memory when compared to the adjacency matrix as long as the graph is sparse. Vertices and adjacency lists are arranged lexicographically to provide consistent output with respect to the graph.

Graph Representation

The graph is represented by a data structure with the graph name, a linked list of vertices, and the total number of vertices and edges as counters. The container is used for all graph operations.

Vertex Representation

Every vertex is associated with a structure that has a distinct string ID, the pointer to the beginning of the respective adjacency list, and a pointer to the next vertex in the linked list representing the whole graph. The vertices can be stored in a lexicographic order, as it is much more convenient for searching and printing purposes.

Edge Representation

An adjacency list node is used to represent each edge that contains a reference to the destination vertex, edge weight, and a reference to the next adjacent vertex. The graph is undirected which means that every edge appears twice in the graph; that is, it has two identifying ends.

Adjacency List Implementation

The adjacency list is implemented through the use of linked lists, where every vertex maintains its own list of neighbor vertices. New edges are entered in lexicographical order to guarantee the correctness while traversing, printing, and performing other tasks involving graphs at the same time allowing for efficient memory usage of sparse graphs.

GRAPH OPERATION IMPLEMENTATION

The graph allows for a number of fundamental operations that facilitate the implementation of various actions on an undirected and weighted graph. These operations comprise addition of vertices and edges, finding the degree of a vertex, checking the presence of an edge, conducting a Breadth First Search (BFS) and Depth First Search (DFS) algorithm, checking the presence of a path between two vertices, finding the Minimum Spanning Tree (MST) of the graph, printing the graph. The optional shortest-path operation is reserved but not implemented in the final version.

Add Vertex

With the Add Vertex operation, the new vertex can be added to the graph keeping the order of the vertices maintained. Primarily, before performing insertion, the software checks whether that vertex is already present in the system so as to avoid repetition in the names. Once the name of the vertex is confirmed as original and the memory has been allocated, the vertex gets linked.

Add Edge

Adding Edge operation establishes an undirected weighted edge between two

current vertices present in the graph. Before performing the task, validation is done to check if both vertices are valid and exist and both vertices are different, checking whether the edge already exists, and validating whether the weight of the edge is valid. Since this is an undirected graph, the adjacency list node is appended to both vertex adjacency lists.

Get Degree

The Get Degree operation computes the degree of a given vertex by moving through its adjacency list and calculating the count of connecting edges. In case the selected vertex is missing, the operation would produce a negative value in response to show that the request cannot be fulfilled.

Edge Check

The Edge Check function confirms the direct connectivity of two vertices through an edge. The function identifies the vertices and scans the source vertex's adjacency list for the neighboring vertex. The result is a true or false answer as to the existence of the edge.

Breadth-First Search

Breadth-First Search (BFS) travels the graph layer-wise using Queues. It starts with a source vertex and explores its neighbouring vertices in the order in which they are discovered while making sure that a vertex is not visited more than once using a visited array.

Depth-First Search

Depth-first search is an algorithm used to traverse the graph starting from a vertex and continuing along one branch until the end is reached. For storage purposes, it

uses the stack data structure and also refers to visited arrays to confirm the vertices that have already been processed. The neighboring vertices are pushed in the stack according to the reverse alphabetical order of their names.

Path Check

The Path Check operation identifies if any path exists between two vertices through the use of Breadth-First Search. The algorithm initiates from a source vertex and goes on searching until the destination vertex is found or no reachable vertices are left. The operation has a Boolean output stating if the path exists.

Minimum Spanning Tree

The procedure of Minimum Spanning Tree (MST) is the making of a network in such a way that all of its vertices are connected with the least edge weight possible. The procedure repeatedly takes the lowest edge available that joins a visited vertex with an unvisited one, until all vertices are visited. The resulting graph is guaranteed to contain at least the least possible number of edges.

Print Graph

The Print Graph operation shows us the graph using a special way of writing it down. It lists all the points on the graph, which are called vertices in an order from A to Z. Then it shows us all the lines that connect these points and these lines are called weighted edges. The thing about this graph is that it does not have a direction so when it shows us the lines that connect the points it only shows each line one time even though it actually has two copies of each line in its lists of connections.

Shortest Path

The shortest-path feature is not active in the final submitted version. Command 9 reads the source and destination vertex names but does not perform a shortest-path computation or produce output. This optional bonus feature was excluded to preserve the stability of the required graph operations.

PROGRAM SPECIFICATIONS

The software is designed to implement undirected weighted graphs and provide a command-based interface that permits various operations on the graph. The vertices of the graph are identified by unique names and edges of the graph are provided with integer weights of the edges and two vertices. Vertices and the adjacency lists are kept in the same lexicographical order in the graph in order to facilitate reading and traversing through it.

The software allows for simple operations on graphs that include vertex addition, edge addition, finding degree of vertex, checking if there is an edge between two vertices, BFS and DFS, checking for existence of a path and finding a minimum spanning tree, and printing the graph structure.

To make sure that the software is functioning properly, it checks the user input for any mistakes before executing the operations on the graph. The software does not allow for creation of duplicate edges and vertices, self-loops are rejected, and the edge weight has to meet the required specifications.

TESTING PROCESS

To validate the accuracy of the implementation, a detailed set of test cases was created and executed. These test cases analyzed various graph operations so as to make sure that all implemented features work properly under different conditions.

The first group of tests is related to vertex operations, such as adding vertex and avoiding inserting duplicate vertices.

The second group of tests is connected to edge operations, including adding edges with weights, preventing duplication of edges, rejecting self-loops, and validating edge weights.

The third group is about graph property operations, degree computation and edge existence checking, which confirm the correctness of the adjacency list representation.

The fourth group has to do with algorithm testing in relation to graph traversals, which include BFS and DFS algorithms. This allows to make sure that all the vertices are accessed properly and no same vertex is accessed multiple times.

The fifth group of tests is concentrating on connectivity operations in terms of path existence checking for different vertex pairs. It confirms the fact that the program is able to differentiate between connected and unconnected vertices.

The sixth category of tests is related to MST generation based on weighted connected

graphs. It verifies that the generated spanning tree connects all vertices while minimizing the total edge weight.

Additionally, extra tests were performed to confirm that the graph was being printed correctly with the vertices and edges appearing in the specified graph notation and lexicographical order.

The final tests were to make sure that the correct errors were thrown for invalid operations that were attempted in the program such as attempts to add the same vertex twice, attempts to add the same edge twice, an edge with a weight less than 0, a self-loop edge, and operations involving vertices that do not exist.

All tests gave the expected outputs after multiple passes of testing, debugging, and modifying the code to ensure that all the functionalities implemented conform to the project guidelines and work as expected.

LIMITATIONS

While the graph application successfully performs the required graph operations, several limitations remain.

First, the application only supports undirected weighted graphs. It does not support directed graphs or other graph representations.

Second, the software assumes that every vertex has a unique name and maintains all vertices and adjacency lists in lexicographical order. The program also does not provide operations for deleting or modifying existing vertices and edges once they have been added.

Third, the implementation relies on adjacency lists and sequential searching when locating vertices and traversing adjacency lists. As the number of vertices and edges increases, search and traversal operations become less efficient compared to implementations that use more advanced indexing or lookup structures.

Fourth, although a heap module was considered during the project's development, it is not used in the final implementation. Consequently, the Minimum Spanning Tree (MST) implementation does not utilize a priority queue, making it less efficient for very large graphs.

Finally, the Shortest Path operation (Command 9) is not implemented in the final submitted version. The command is recognized by the program, but no shortest-path computation or output is produced. As a result, shortest-path queries are not supported in this implementation.

CONCLUSION

The project was successfully implemented by applying the basic concepts of graphs with the help of undirected weighted graph and adjacency list. The project was able to highlight and demonstrate what data structures and graph algorithms can do. It achieved the implementation of linked lists and utilized stacks and queues effectively to perform the core graph operations such as adding vertices and edges, calculating degrees, checking edges, traversing graphs, confirming paths, displaying graphs, and generating Minimum Spanning Tree (MST).

It was shown how data structures work to ensure that graph data is represented and managed properly. The project incorporated stacks and queues to implement DFS and BFS, respectively. The adjacency lists offered an appropriate option for representing information about graph connections. The test showed that all the operations performed correctly and all outcomes were given as expected.

To sum up, the project completed the tasks it had to complete and it developed the graph application with all the specified requirements according to CCDSALG MCO2. The implementation increased the knowledge of graph concepts such as the ways of representing graphs, traversing graphs, creating minimum spanning trees, and combining different data structures. The project was very useful in terms of developing programming skills and understanding graph algorithms through practical implementation.

REFERENCES

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Weiss, M. A. (2014). Data Structures and Algorithm Analysis in C (2nd ed.). Pearson.

Fernandez, R. A. (2026). CCDSALG Course Materials and Lecture Notes. De La Salle University.

APPENDIX A - RECORD OF CONTRIBUTION

Activity	Catera	Latido	Medran
Stack Implementation	33.3	33.3	33.3
Queue Implementation	33.3	33.3	33.3
Heap Implementation	33.3	33.3	33.3
Basic Graph Implementation	33.3	33.3	33.3
Add Vertex	30	40	30
Add Edge	27.5	45	27.5
Degree Check	30	40	30
Edge Check	30	40	30
BFS	33.3	33.3	33.3
DFS	33.3	33.3	33.3
Path Check	30	40	30
MST	33.3	33.3	33.3
Shortest Path	33.3	33.3	33.3
Testing	30	40	30
Documentation	33.3	33.3	33.3
Raw Total	477.2	544.7	477.2
TOTAL	31.83%	36.34%	31.83%

APPENDIX B - AI USE DECLARATION

Task	N	S	R	G
Understanding Specifications		/		
Understanding Algorithm or References		/		
Stack Implementation	/			
Queue Implementation	/			
Expression Conversion	/			
Expression Evaluation	/			
Documentation		/		

N = No AI Assistance

S = Summarization

R = Research / Reference

G = Content Generation

AI Tools Used

1. ChatGPT

STATEMENT OF ACCOUNTABILITY

We affirm that all AI-assisted contributions have been critically examined by the project group. We acknowledge the limitations of Generative AI systems, including possible errors or biases, and accept full accountability for all content in this project and documentation.